

实验报告

数字逻辑与计算机组成实验

lab03: 计时器与电子时钟

姓名: 法童

学号: 251250046

邮箱: 251250046@smail.nju.edu.cn

实验时间: 2026-09-16

1 实验目的

1. 掌握参数化分频器的设计：把板载 100 MHz 时钟分频为 1 Hz 的计时时钟与 1 kHz 的扫描时钟，理解翻转计数法分频的频率计算。
2. 用 1 Hz 时钟驱动一个模 60 计数器，实现开始、暂停、清零三种控制，并在两位七段数码管上以十进制显示 00–59；计满一周时用 LED 闪烁一个时钟周期提示。
3. 在拓展部分实现多功能电子时钟：时/分/秒走时、调时（选位 + 加减）、闹钟（定时 LED 闪烁）、百分秒精度秒表（可停止重启），掌握多模块层次化设计、按键消抖与多位动态扫描显示。
4. 对比“分频时钟”与“使能脉冲（tick）”两种时基产生方式，理解单时钟域设计在工程上的优势；继续熟练 top + .xdc 的上板流程。

2 实验原理

2.1 计时器 (3.5.1)

题目要求先用 100 MHz 时钟产生 1 Hz 的时钟信号，再以该 1 Hz 信号作为计时时钟计数。分频采用**翻转计数法**：用一个计数器对输入时钟计数，每计满 N 个周期就把输出电平翻转一次，于是输出为方波，周期为 $2N$ 个输入周期，即

$$N = \frac{f_{\text{clk}}}{2f_{\text{out}}} - 1, \quad f_{\text{out}} = \frac{100 \text{ MHz}}{2(N + 1)}.$$

取 $f_{\text{out}} = 1 \text{ Hz}$ 得 $N = 49\,999\,999$ ，输出周期 1 s、占空比 50%。题目中“闪烁一个时钟周期”即指该 1 Hz 时钟的一个周期，也就是 1 s。

模 60 计数器在 1 Hz 时钟上升沿累加，取值 0–59：计到 59 的下一拍回到 0，同时把进位输出 rco 拉高一个时钟周期，用于点亮 LED 提示计时结束。计数器输出 Q 是 6 位二进制，需要拆成十位与个位两个 BCD 码 ($Q/10$ 与 $Q\%10$) 分别译码，才能在数码管上以十进制显示。

Nexys A7 的八位数码管为**共阳极、段选与位选均低有效**：段码位为 0 表示该段点亮，位选 AN 为 0 表示该位选通。两位数字无法同时显示，因此用 1 kHz 时钟在两位之间来回切换位选，配合视觉暂留即可看到稳定的两位数。

2.2 电子时钟 (3.5.2)

拓展部分需要在一套硬件上同时实现走时、调时、闹钟与秒表，共同点是都需要“以秒（或更细）为单位推进”的节拍，区别在于各自的**使能、复位与进位关系不同**。因此设计分三层：

时基层：仍用 100 MHz 主时钟，但不再分频出新时钟，而是产生**单周期的使能脉冲**（tick）——计数器计满一个额定周期后只在某一个 100 MHz 周期内输出高电平。所有计数逻辑都在 100 MHz 单时钟域内工作，只靠 tick 决定“何时该走一步”，避免多时钟域与门控时钟带来的时序问题。

计数层：用参数化的两位 BCD 加减计数器作为基本单元，支持加计数与减计数，并输出进位 carry 与借位 borrow，把若干单元级联成两条计数链：

- 走时链：秒(00–59)→分(00–59)→时(00–23),级联信号取上一位的 carry/borrow;
- 秒表链：百分秒 (00–99) → 秒 (00–59) → 分 (00–59), 百分秒由 10 ms 使能驱动, 100 个百分秒进位到 1 秒。

功能层：调时把按键产生的加/减脉冲注入对应计数单元的 en_up/en_down; 闹钟比较当前时与开关设定的时, 相等且分为 00 时让 LED 闪烁; 显示源选择用模式开关在两条链之间切换。

3 实验环境与器材

- 开发板：Nexys A7-100T (FPGA 型号 xc7a100tcsg324-1)
- 开发工具：Vivado 2020.2 (非工程模式, build.bat / prog.bat 脚本驱动)
- 硬件描述语言：Verilog HDL

4 设计思路

4.1 计时器：分频器、模 60 计数器与两位扫描显示

计时器工程共四个模块：clkgen (参数化分频器)、timer (模 60 计数器)、display (两位扫描与位选译码)、bcd7seg (BCD 转七段段码), 由 top 包装到板级端口。系统框图与端口分配如下：

分频器是参数化的, 同一段代码通过 clk_freq 参数复用出 1 Hz 与 1 kHz 两路时钟; 另设 clken 允许暂停分频:

```

1 module clkgen(input clkkin, input rst, input clken, output reg clkout)
  ;
2     parameter clk_freq=1000;
3     parameter countlimit=100000000/2/clk_freq-1;
4     reg [31:0] clkcount;

```

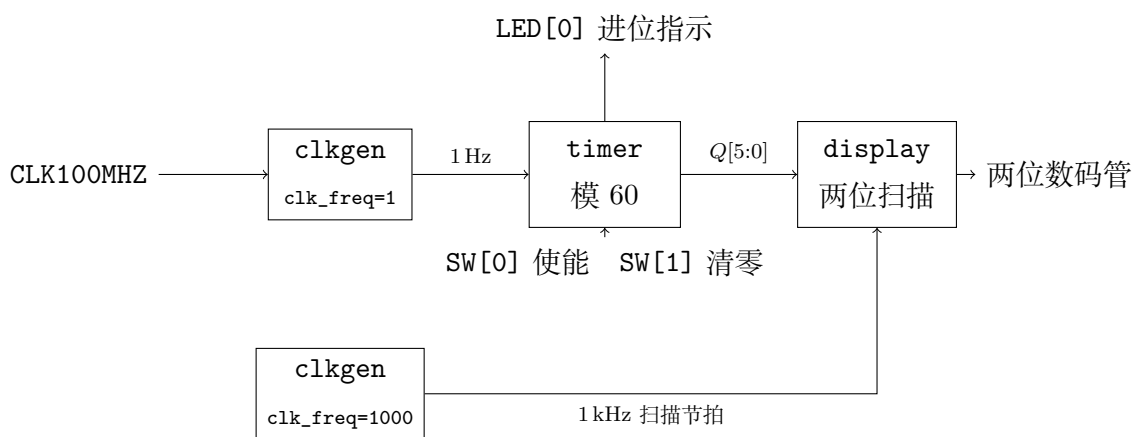


图 1: 计时器系统框图

表 1: 计时器板级端口分配

板级资源	顶层信号	功能
CLK100MHZ	CLK100MHZ	100 MHz 系统时钟
SW[0]	en	开始 (1) / 暂停 (0)
SW[1]	rst	计数器清零
LED[0]	rco	计满 60 时亮一个 1 Hz 时钟周期
AN[7:0]、CA..CG、DP	数码管	两位十进制显示；未用位熄灭、小数点熄灭

```

5      always @ (posedge clkin)
6          if(rst) begin clkcount<=0; clkout<=1'b0; end
7          else if(clken) begin
8              if(clkcount>=countlimit) begin clkcount<=32'd0; clkout<=~
9                  clkout; end
10             else clkcount<=clkcount+1;
11         end
endmodule

```

模 60 计数器只有十几行，要点是**清零优先级最高**、计满回卷并置起进位：

```

1  always @(posedge clk)
2      if (rst) begin                                // 清零优先于使能：暂停时也
3          Q <= 0; rco <= 1'b0;                      能清零
4      end
5      else if (en)
6          if (Q==59) begin                          // 00..59 共用 60 个状态
7              Q <= 0; rco <= 1'b1;                  // 回卷并给出一个周期的进位
8              脉冲
9          end
10         else begin Q <= Q+1; rco <= 1'b0; end

```

display 用除法与取模把 Q 拆成十位、个位后分别译码（组合逻辑），再用扫描时钟在两位之间切换：

```

1  assign out_dis0 = Q%10;    assign out_dis1 = Q/10;    // 个位 / 十位
2  bcd7seg(out_dis0, h0);    bcd7seg(out_dis1, h1);    // 两路段码
3  always @(select)
4      case (select)
5          1'b0: begin seg7[7:0] = 8'b11111110; h=h0; end // 只选通右
6                  位（个位）
7          1'b1: begin seg7[7:0] = 8'b11111101; h=h1; end // 只选通左
6                  位（十位）
7      endcase

```

其中 `select` 直接接 1kHz 扫描时钟，位选每 0.5ms 切换一次、一轮 2 位共 1ms，刷新率 1kHz，远高于人眼分辨能力，因此看到的是两位同时点亮。段码表按共阳极低有效编写，`default` 输出全 1（熄灭），因此送入 10–15 时该位不显示。

4.2 电子时钟：时基、计数链、控制与显示

电子钟工程的层次如图 2: tickgen 产生全部时基, debounce 处理按键, set_signal/sw_signal 把按键脉冲翻译成控制信号, eclock 是功能核心, bcd_display6 负责 6 位动态扫描。

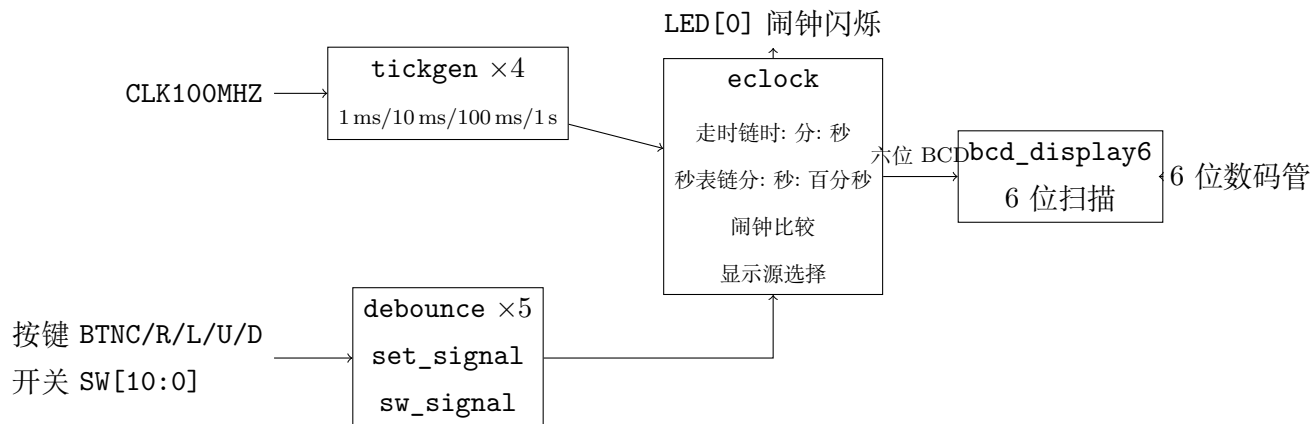


图 2: 电子钟系统框图

(1) 时基：单周期使能脉冲

tickgen 与 clkgen 的区别在于：计满额定周期时只在一个主时钟周期内把 tick 拉高，而不是翻转输出电平。

```

1 localparam integer countlimit = clk_freq / tick_freq - 1;
2 always @(posedge clk)
3     if (rst) begin clkcount <= 32'd0; tick <= 1'b0; end
4     else if (en)
5         if (clkcount >= countlimit) begin
6             clkcount <= 32'd0; tick <= 1'b1;          // 只高一拍
7         end
8         else begin clkcount <= clkcount + 32'd1; tick <= 1'b0; end

```

四个实例共用一段代码，靠参数得到全部节拍，见表 2。

表 2: tickgen 时基分配（主时钟 100 MHz）

实例	tick_freq	计数上限	用途
u_tick_1ms	1000	99 999	数码管动态扫描节拍
u_tick_10ms	100	999 999	秒表百分秒计数、按键消抖采样
u_tick_100ms	10	9 999 999	闹钟 LED 闪烁节拍
u_tick_1s	1	99 999 999	时钟走时节拍

(2) 计数单元：参数化两位 BCD 加减计数器

bcd_count2 是整套设计的复用核心：两位 BCD (bcd1 为高位) 加上限 max1/max0 参数，同时支持加计数与减计数，并输出进位、借位。加计数到上限回卷给出 carry，减计数到 00 回卷到上限给出 borrow：

```

1 parameter [3:0] max0 = 9; // 例化时改为 (max1,
   max0):
2 parameter [3:0] max1 = 9; // (5,9)=00..59
   (2,3)=00..23 (9,9)=00..99
3 always @(posedge clk)
4     if (rst) begin bcd0<=0; bcd1<=0; carry<=0; borrow<=0; end
5     else if (en_up) begin
6         borrow <= 1'b0;
7         if (bcd0==max0 && bcd1==max1) begin // 计满回卷
8             bcd0<=4'd0; bcd1<=4'd0; carry<=1'b1;
9         end
10        else if (bcd0==4'd9) begin // 个位向十位进位
11            bcd0<=4'd0; bcd1<=bcd1+4'd1; carry<=1'b0;
12        end
13        else begin bcd0<=bcd0+4'd1; carry<=1'b0; end
14    end
15    else if (en_down) begin // 减计数：00 回
   卷到上限并借位
16        ...
17    end

```

只要换一个 max1/max0 组合，同一模块就变成 00-59 的秒/分、00-23 的时、00-99 的百分秒，共例化 6 次。走时链的级联如下（秒与分的上限为 59，时上限为 23）：

```

1 bcd_count2 #(.max1(5), .max0(9)) u_clk_sec ( // 00..59
2     .clk(clk), .rst(rst), .en_up(tick_1s && en || up_s), .en_down(
   down_s),
3     .bcd0(clk_s0), .bcd1(clk_s1), .carry(carry_s2m), .borrow(
   borrow_s2m));
4 bcd_count2 #(.max1(5), .max0(9)) u_clk_min ( // 分的加来自秒的进
   位，也可由调时按键驱动
5     ... .en_up(carry_s2m || up_m), .en_down(borrow_s2m || down_m),
6     .carry(carry_m2h), .borrow(borrow_m2h));
7 bcd_count2 #(.max1(2), .max0(3)) u_clk_hour ( // 00..23

```

```
8 ... .en_up(carry_m2h || up_h), .en_down(borrow_m2h || down_h));
```

秒表链与走时链结构相同, 但用独立的复位/使能 (rst_sw、en_sw), 百分秒用 10 ms 节拍:

```
1 bcd_count2 #(.max1(9), .max0(9)) u_sw_frac ( // 00..99 百分秒
2     .clk(clk), .rst(rst_sw), .en_up(tick_10ms && en_sw), ...);
3 bcd_count2 #(.max1(5), .max0(9)) u_sw_sec ( ... .en_up(carry_f2s) ...
4     );
5 bcd_count2 #(.max1(5), .max0(9)) u_sw_min ( ... .en_up(carry_s2m_sw)
6     ... );
```

(3) 调时与秒表控制

调时需要一个”当前调整对象”状态: 按 BTNL 在秒/分/时之间循环, set_signal 用 2 位 set 寄存器记住位置, 再把 BTNU/BTND 的加/减脉冲分发到对应单元:

```
1 always @(posedge clk)
2     if (set_edge) set <= set==2'd2 ? 2'd0 : set+2'd1; // 秒 -> 分
3     // -> 时 循环
4 assign up_s = mode==1'b0 && set==2'd0 && up_edge; // 只在时钟
5     模式下生效
6 assign down_s = mode==1'b0 && set==2'd0 && down_edge;
7 // ... up_m/down_m、up_h/down_h 同理
```

秒表的启停/清零是独立状态, sw_signal 用 BTNC 翻转一个使能位、BTNR 复位该位; 按键抖动由 debounce 消除——用 10 ms 节拍连续采样两次都一致才认为按键稳定, 再对稳定电平做上升沿检测, 输出**单周期**的边沿脉冲:

```
1 always @(posedge clk)
2     if (tick_10ms) begin delay1 <= btn; delay2 <= delay1; end // 10
3     ms 采样两次
4 wire btn_stable = delay1 & delay2; // 连
5     续两次为高才认键
6 always @(posedge clk) stable_delay <= btn_stable;
7 assign btn_edge = btn_stable & ~stable_delay; // 上
8     升沿 -> 单周期脉冲
```

(4) 闹钟与显示源选择

闹钟比较当前时与开关设定的闹钟时 (SW[10:7] 为小时十位、SW[6:3] 为小时个位), 相等且当前分为 00 时, 让 LED 跟随 100ms 节拍翻转 (5Hz 闪烁):

```

1 always @(posedge clk)
2     if (tick_100ms) blink <= ~blink;           // 100ms 翻转一次 ->
3         5Hz
4 always @(posedge clk)
5     if (mode==0 && en_alarm && alarm_bcd0==clk_h0 &&
6         alarm_bcd1==clk_h1 && clk_m0 == 0 && clk_m1 == 0)
7         alarm <= blink;                         // 到点则闪烁
8     else
9         alarm <= 0;

```

显示源用模式开关在两条链之间二选一, 清零时统一送 10 (bcd7seg 的越界值, 输出全灭), 因此复位后数码管全灭:

```

1 assign bcd0 = !rst ? (mode ? sw_f0 : clk_s0) : 4'd10; // 最右位: 秒
2             个位 / 百分秒个位
3 assign bcd1 = !rst ? (mode ? sw_f1 : clk_s1) : 4'd10;
4 assign bcd2 = !rst ? (mode ? sw_s0 : clk_m0) : 4'd10; // 秒表: 秒个
5             位 / 时钟: 分个位
6 assign bcd3 = !rst ? (mode ? sw_s1 : clk_m1) : 4'd10; // 秒表: 秒十
7             位 / 时钟: 分十位
8 assign bcd4 = !rst ? (mode ? sw_m0 : clk_h0) : 4'd10; // 秒表: 分个
9             位 / 时钟: 时个位
10 assign bcd5 = !rst ? (mode ? sw_m1 : clk_h1) : 4'd10; // 秒表: 分十
11              位 / 时钟: 时十位

```

于是时钟模式下六位从左到右显示”时时分分秒秒”, 秒表模式下显示”分分秒秒百分秒百分秒”。

(5) 六位动态扫描显示

bcd_display6 把 6 个 bcd7seg 实例的段码接到一个多路选择器上, 位选计数器由 1ms 节拍推进, 扫描一位 1ms、一轮 6ms, 刷新率约 167Hz:

```

1 always @(posedge clk)
2     if (rst) select <= 3'd0;
3     else if (en) select <= (select == 3'd5) ? 3'd0 : select + 3'd1;
4         // 0..5 循环

```

```

4 always @(*)
5     case (select)
6         3'd0: begin seg7 = 8'b11111110; h = h0; end // 只选通 AN[0]
              (最右位)
7         3'd1: begin seg7 = 8'b111111101; h = h1; end
8         // ... 依次右移选通位
9         default: begin seg7 = 8'b11111111; h = 7'b11111111; end
10    endcase

```

(6) 板级端口分配

顶层把时钟、按键、开关与数码管接到功能模块上，各控制量的分配见表 3。

表 3: 电子钟开关与按键功能分配

板级资源	信号	功能
CPU_RESETN	rst	全局清零（低有效，代码内取反成高有效）
SW[1]	en	时钟走时使能
SW[0]	mode	显示模式：0 = 时钟，1 = 秒表
SW[2]	en_alarm	闹钟使能
SW[10:7]	alarm_bcd1	闹钟小时十位
SW[6:3]	alarm_bcd0	闹钟小时个位
BTNL	set_edge	选择调整对象：秒 → 分 → 时循环
BTNU / BTND	up_edge / down_edge	调整对象加 1 / 减 1
BTNC	en_edge	秒表启动/暂停（每按一次翻转）
BTNR	rst_edge	秒表清零
LED[0]	alarm	闹钟到点闪烁
AN[7:0]、CA..CG、DP	数码管	6 位十进制显示；未用位与小数点熄灭

5 实验过程

1. 编写计时器各模块：参数化分频器 `clkgen.v`、模 60 计数器 `timer.v`、两位扫描显示 `display.v` 与 `bcd7seg.v`，再按 top 包装模式编写 `top.v`，把逻辑端口映射到 SW/LED/AN。
2. 编写约束文件 `timer.xdc`：从根目录 `nexysa7.xdc` 复制，仅取消本次用到引脚的注释（SW[0]、SW[1]、LED[0]、AN[0..7]、CA..CG、DP、CLK100MHZ）；未使用的 AN[7:2] 在 `top.v` 中置 1（熄灭）、DP 置 1。

3. 运行 `build.bat lab03\timer` 完成综合、实现，生成比特流 `build/top.bit`；运行 `prog.bat lab03\timer` 下载到开发板，测试开始/暂停/清零与进位闪烁。
4. 编写电子钟各模块：`tickgen.v`（时基）、`bcd_count2.v`（参数化 BCD 加减计数器）、`debounce.v`（按键消抖）、`bcd_display6.v`（六位扫描），再由 `eclock.v` 例化两条计数链、闹钟比较与显示源选择（内部另含 `set_signal`、`sw_signal` 两个控制模块）。
5. 编写 `eclock.xdc`：在计时器约束基础上增加 `CPU_RESETN`、按键 `BTNC/BTNU/BTNL/BTNR/BTND` 与 `SW[10:0]` 的约束；其中 `SW[8]`、`SW[9]` 属于 bank 34，保持 `LVC MOS18` 不变。
6. 运行 `build.bat lab03\eclock` 生成比特流并 `prog.bat` 下载，逐项验证走时、调时、闹钟与秒表功能。

```

lab03 > timer > vivado.log
1 #-----
2 # Vivado v2020.2 (64-bit)
3 # SW Build 3064766 on Wed Nov 18 09:12:45 MST 2020
4 # IP Build 3064653 on Wed Nov 18 14:17:31 MST 2020
5 # Start of session at: Wed Sep 16 10:59:09 2026
6 # Process ID: 16096
7 # Current directory: E:\NJU-DLCoexp-2026-Fall\lab03\timer
8 # Command line: vivado.exe -mode batch -source E:\NJU-DLCoexp-2026-Fall\scripts\download.tcl
9 # Log file: E:\NJU-DLCoexp-2026-Fall\lab03\timer\vivado.log
10 # Journal file: E:\NJU-DLCoexp-2026-Fall\lab03\timer\vivado.jou
11 #-----
12 source {E:\NJU-DLCoexp-2026-Fall\scripts\download.tcl}
13 # open_hw_manager
14 # connect_hw_server
15 INFO: [Labtools 27-2285] Connecting to hw_server url TCP:localhost:3121
16 INFO: [Labtools 27-2222] Launching hw_server...
17 INFO: [Labtools 27-2221] Launch Output:
18
19 ***** Xilinx hw_server v2020.2
20 ***** Build date : Nov 18 2020 at 10:01:48
21 ***** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.
22
23
24 INFO: [Labtools 27-3415] Connecting to cs_server url TCP:localhost:3042
25 INFO: [Labtools 27-3417] Launching cs_server...
26 INFO: [Labtools 27-2221] Launch Output:
27
28
29 ***** Xilinx cs_server v2020.2
30 ***** Build date : Nov 04 2020-05:02:56
31 ***** Build number : 2020.2.1604437376
32 ***** Copyright 2017-2020 Xilinx, Inc. All Rights Reserved.
33
34
35
36 connect_hw_server: Time (s): cpu = 00:00:00 ; elapsed = 00:00:10 . Memory (MB): peak = 1020.301 ; gain = 24.848
37 # open_hw_target
38 INFO: [Labtools:tcl 44-466] Opening hw_target localhost:3121/xilinx_tcf/Digilent/210292B618E3A
39 # set_device [lindex [get_hw_devices] 0]
40 # set_property PROGRAM.FILE {./build/top.bit} $device
41 # program_hw_devices $device
42 INFO: [Labtools 27-3164] End of startup status: HIGH
43 # close_hw_manager
44 ***** WebTalk v2020.2 (64-bit)
45 ***** SW Build 3064766 on Wed Nov 18 09:12:45 MST 2020
46 ***** IP Build 3064653 on Wed Nov 18 14:17:31 MST 2020
47 ***** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.
48
49 source E:\NJU-DLCoexp-2026-Fall\lab03\timer\Xil/Vivado-16096-DESKTOP-GGK86J5/webtalk/labtool_webtalk.tcl -notrace
50 INFO: [Common 17-206] Exiting Webtalk at Wed Sep 16 10:59:21 2026...
51 INFO: [Common 17-206] Exiting Vivado at Wed Sep 16 10:59:21 2026...
52

```

图 3: 计时器工程综合与实现完成

6 测试方法

本实验的验证以上板实测为主，辅以关键信号的逻辑推导，不使用课程 OJ 判题作为验证手段。之所以不以功能仿真为主要手段，是因为两个设计的时基都由 100 MHz 分

```

237 | |BlackBox name |Instances |
238 +-----+-----+
239 +-----+-----+
240
241 Report Cell Usage:
242 +-----+-----+
243 | |Cell |Count |
244 +-----+-----+
245 |1 |BUFG | 1|
246 |2 |CARRY4 | 32|
247 |3 |LUT1 | 11|
248 |4 |LUT2 | 27|
249 |5 |LUT3 | 22|
250 |6 |LUT4 | 69|
251 |7 |LUT5 | 35|
252 |8 |LUT6 | 108|
253 |9 |FDRE | 209|
254 |10 |IBUF | 18|
255 |11 |OBUF | 17|
256 +-----+-----+
257
258 Finished Writing Synthesis Report : Time (s): cpu = 00:00:11 ; elapsed = 00:00:12 . Memory (MB): peak = 1017.719 ; gain = 0.000
259
260 Synthesis finished with 0 errors, 0 critical warnings and 0 warnings.
261 Synthesis Optimization Runtime : Time (s): cpu = 00:00:07 ; elapsed = 00:00:11 . Memory (MB): peak = 1017.719 ; gain = 0.000
262 Synthesis Optimization Complete : Time (s): cpu = 00:00:11 ; elapsed = 00:00:12 . Memory (MB): peak = 1017.719 ; gain = 0.000
263 INFO: [Project 1-571] Translating synthesized netlist
264 Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.002 . Memory (MB): peak = 1017.719 ; gain = 0.000
265 INFO: [Netlist 29-17] Analyzing 32 Unisim elements for replacement
266 INFO: [Netlist 29-28] Unisim Transformation completed in 0 CPU seconds
267 INFO: [Project 1-570] Preparing netlist for logic optimization
268 Parsing XDC File [E:/NJU-DLCOexp-2026-Fall/lab03/eclock/eclock.xdc]
269 Finished Parsing XDC File [E:/NJU-DLCOexp-2026-Fall/lab03/eclock/eclock.xdc]
270 INFO: [Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).
271 Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 1017.719 ; gain = 0.000
272 INFO: [Project 1-111] Unisim Transformation Summary:
273 No Unisim elements were transformed.

```

图 4: 电子钟工程综合与实现完成

频/分频得到：计时器要观察到一次进位需要 60s 的逻辑时间（约 6×10^9 个主时钟周期），电子钟走到分/时进位更要 10^4 s 量级，直接仿真的时间与存储代价都过高；若要做短时仿真，可行的做法是把分频参数改小（例如把 `clk_freq` 改成 1000，使 1Hz 变成 1ms）后再观察计数链行为。本次改为在板上直接验证。

测试信号的选择遵守“覆盖边界 + 组合控制”的原则，逐项设计如下。

计时器（测试信号为 SW[0]、SW[1] 的组合与计数值的边界）：

- **清零**：拨 SW[1]，数码管应显示 00，且进位 LED 熄灭；
- **开始/暂停**：SW[1]=0、SW[0]=1，数码管应从 00 逐秒递增；中途把 SW[0] 拨回 0，数字应停住不动，再拨回 1 应从暂停处继续（而不是从 00 重来）；
- **进位边界**：重点观察 09→10（个位向十位进位）与 59→00（整体回卷）两处，后者应看到 LED 点亮一个 1Hz 时钟周期（即亮约 1s）；
- **清零优先级**：先暂停（SW[0]=0），再拨 SW[1]，计数器仍应从暂停状态被清零——这正是本次修掉的一个优先级问题。

电子钟（测试信号为各按键/开关的单独遍历与时间边界）：

- **走时与进位**：CPU_RESETN 清零后置 SW[1]=1，用手机秒表比对 1s 节拍，重点观察 59 秒 → 进分、59 分 → 进时、23 时 → 回卷到 00 时三处边界；
- **调时**：按 BTNL 逐次切换调整对象（秒/分/时），每档按 BTNU/BTND 各若干次，验证加/减正确、位与位之间不串扰；再验证 00 减 1 应回卷到 59（秒/分）或 23（时）；

- **闹钟**：把 SW[10:7]/SW[6:3] 设为当前小时，SW[2]=1，等到分钟回到 00 时观察 LED[0] 是否以约 5 Hz 闪烁；再把闹钟设为别的小时，确认 LED 不亮；
- **秒表**：SW[0]=1 切入秒表模式，按 BTNC 启动，与手机秒表比对；再按 BTNC 应停住且不清零（暂停），再按应继续；按 BTNR 应清零为全 0；
- **独立性**：在秒表暂停状态下确认时钟链仍在正常走时（模式开关切回 0 观察），验证两条链的使能/复位互不影响。

7 实验结果

7.1 计时器 (3.5.1)

下载比特流后上板实测，现象与设计预期一致：

- **清零**：拨 SW[1]，两位数码管显示 00，LED[0] 熄灭；
- **开始与暂停**：SW[1]=0、SW[0]=1 时数码管从 00 逐秒递增；把 SW[0] 拨回 0，显示停在当前值；再拨回 1 从该值继续，未丢失计数；
- **两位数扫描**：两位数字同时稳定显示、无可见闪烁，个位为右位、十位为左位；
- **进位边界**：09→10 时十位正确加 1；59→00 时数码管回到 00，同时 LED[0] 点亮约 1s（一个 1Hz 时钟周期）后熄灭，即“计时结束闪烁一个时钟周期”的现象；
- **清零优先级**：在暂停（SW[0]=0）状态下拨 SW[1]，计数器仍被清零为 00。

7.2 电子时钟 (3.5.2)

电子钟上板实测各功能均正常：

- **走时**：按 CPU_RESETN 后数码管全灭；置 SW[1]=1 开始走时，六位从左到右显示“时时分分秒秒”。与手机秒表比对一分钟，误差不可察觉；观察 59 秒 → 进分、59 分 → 进时、23 时 → 回卷 00 时三处边界均正确进位；
- **调时**：按 BTNL 可在秒 → 分 → 时之间循环选择调整对象，再用 BTNU/BTND 对当前对象加/减，每按一次恰好变化 1，无一次多跳；在 00 处减 1 会回卷到 59（秒/分）或 23（时）；
- **闹钟**：把闹钟时设为当前小时、SW[2]=1，当分钟回到 00 时 LED[0] 开始以约 5 Hz 闪烁；设为其他小时则不闪烁，SW[2]=0 时任意时刻均不闪烁；

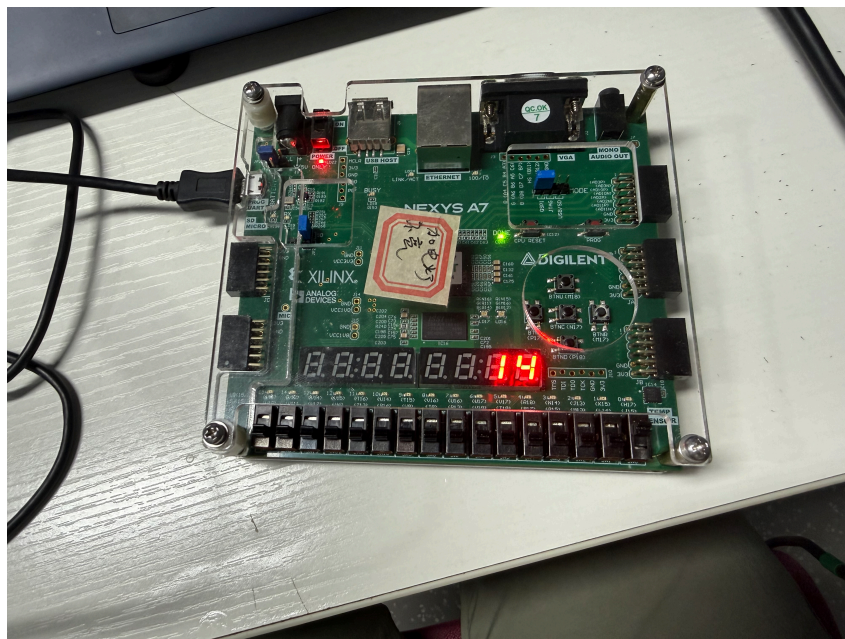


图 5: 计时器上板实测 (计数 / 暂停 / 清零 / 进位闪烁)

- **秒表**: SW[0]=1 切入秒表模式后显示”分分秒秒百分秒百分秒”，按 BTNC 启动后百分秒以肉眼可见的速度滚动，与手机秒表比对一致；再按 BTNC 数字停住（暂停），再按继续累加，按 BTNR 清零为全 0；
- **独立性**: 秒表暂停期间切回时钟模式，时钟链仍在正常走时，说明两条链的使能与复位互不干扰。

8 问题与解决办法

- **数字 9 显示错误**: 段码表中 9 误写成 7'b0000100，该值只让 C 段熄灭、其余段全亮，数码管上呈现出缺少右上竖段的畸形字符。原因：段码位序为 $\{g, f, e, d, c, b, a\}$ 且低有效，手写段码时把段位对错了。解决：改为 7'b0010000（只熄灭 E 段），并把该模块从 display.v 中独立成 bcd7seg.v，同时修正了 lab01/lab02 中的同一处错误。
- **暂停时清零失效**: timer.v 原来把复位写在使能分支内部 (if (en) begin if (rst) ...)，于是 en=0 时复位信号根本不起作用——暂停后拨清零开关没有任何反应，且清掉的进位标志还会残留。解决：把 rst 提到**最高优先级**判断，结构改为 if (rst) ... else if (en) ...，清零在任何状态下都有效。
- **按键抖动导致一次按下触发多次**: 机械按键按下/松开瞬间有数毫秒抖动，直接做边沿检测会在一次操作中产生多个脉冲，表现为调时一次跳好几位、秒表启停不

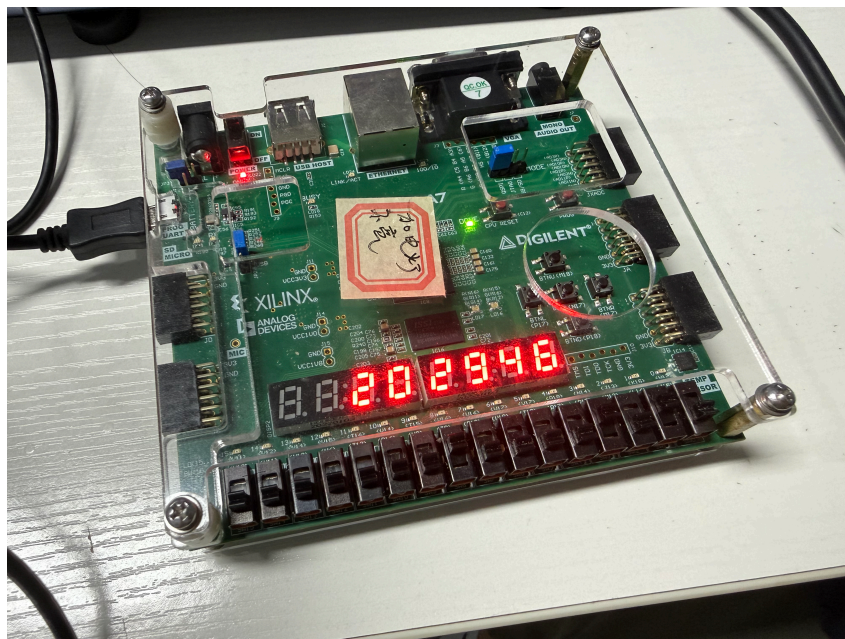


图 6: 电子钟上板实测：走时（时：分：秒）



图 7: 电子钟上板实测：调时

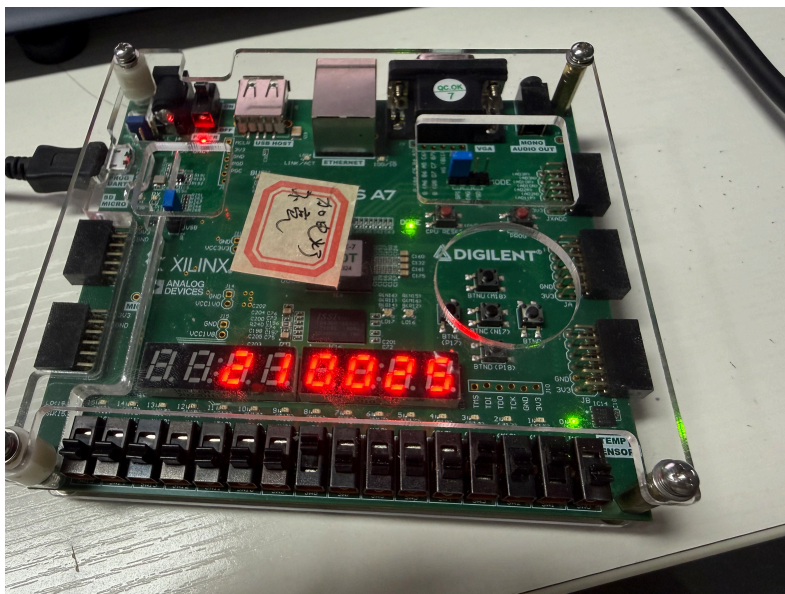


图 8: 电子钟上板实测：闹钟 LED 闪烁

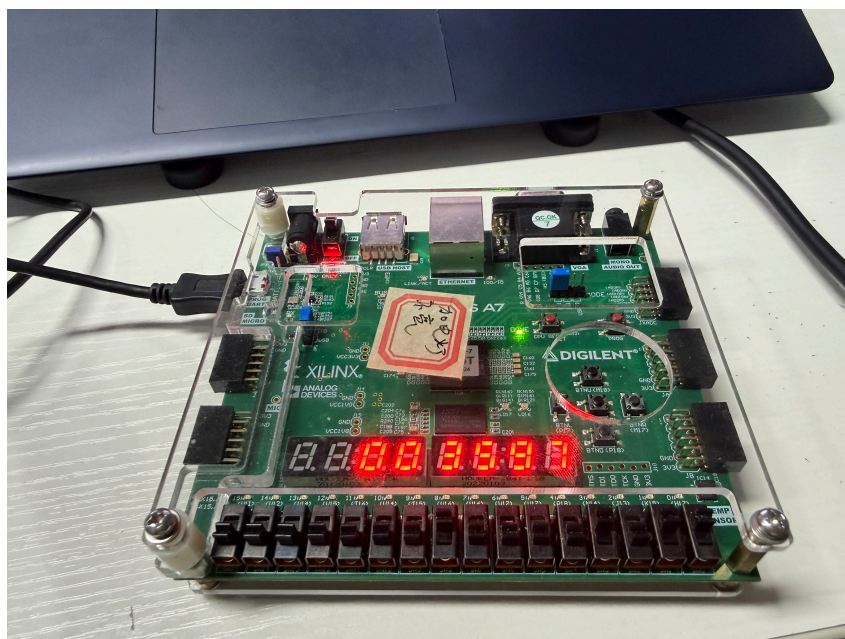


图 9: 秒表上板实测：启停 / 清零 / 百分秒

受控。解决：增加 `debounce` 模块，用 10 ms 节拍连续采样两次且都一致才认定按键稳定，再对稳定电平做上升沿检测，输出单周期脉冲；消抖时间约 20 ms 且不引入新时钟域。

- **秒表的清零/启停不能与时钟共用信号**：最初秒表直接共用全局 `rst` 和走时使能 `en`，导致暂停秒表会把整只时钟一起停掉、清零秒表也会清掉当前时间。解决：把控制信号拆开——`rst` 只做全局清零，`en` 只控制时钟走时，秒表另设 `rst_sw/en_sw`（由 `sw_signal` 模块从 `BTNR/BTNC` 产生），两条链彼此独立。
- **多个计数器共用一个模块的问题**：时（00–23）、分/秒（00–59）、百分秒（00–99）的进制各不相同，若各写一个计数器代码量成倍增长。解决：把上限做成 `max1/max0` 参数，并同时实现加、减计数与进位/借位输出，一处代码例化 6 次即覆盖全部计数单元，调时所需的“减到 00 回卷”也由 `borrow` 链自动完成。
- **让计数随时钟走而不用多套时钟**：若沿用时器的做法为秒表、扫描、闹钟各分频出一路时钟，就会出现多个时钟域和门控时钟。解决：电子钟统一用 100 MHz 主时钟，由 `tickgen` 产生 1 ms/10 ms/100 ms/1 s 四种单周期使能脉冲，各计数链只在对应节拍上走一步，全部逻辑处于单时钟域。
- **六位数码管的引脚与资源复用**：6 位数字若各自独立驱动需要 $6 \times 7 = 42$ 根段线。解决：用 `bcd_display6` 做动态扫描——6 个 `bcd7seg` 译码结果共享一组段选，位选由 3 位计数器循环推进，每 1 ms 换一位，总共只用 7 根段线加 8 根位选线。

9 启示与建议

启示：

1. 分频器与使能脉冲是产生时基的两种不同思路：前者输出一个新时钟（计时器题目明确要求），后者在单时钟域内给出“什么时候走一步”。后者不引入新时钟域、不需要额外的时钟约束，在多功能设计中更稳妥，本实验正好各用了一次、体会了两者的差别。
2. 模块的参数化能显著降低重复代码：`clkgen` 一个参数给出 1 Hz 与 1 kHz，`bcd_count2` 一组上下限参数覆盖 00–23/00–59/00–99 三种进制，`tickgen` 一个参数给出四路节拍。
3. 层次化设计让复杂功能变得可控：电子钟拆成“时基 → 计数 → 控制 → 显示”四层后，各模块都能单独读懂和测试，顶层只是连线。

4. 按键消抖、复位优先级、位选/段码的有效极性这些”细节”才是上板最容易出问题的地方——本次三个真实问题（数字 9 段码、暂停时清零失效、按键抖动）都属于这一类，仿真波形上看不出来，必须在板上才能发现。
5. 动态扫描是数码管扩展位数的通用手段：靠刷新率远高于人眼分辨能力（本设计约 167 Hz）来实现”同时显示”。

建议：目前的显示未使用小数点 DP，可以把 DP 接出来作为时/分/秒之间的分隔符并让它按 1 Hz 闪烁，读数会更像一只真实的电子钟；此外也可用 PWM 驱动板载蜂鸣器替代 LED 闪烁，提示效果更明显。